CAMPUS DIGITAL FP

# Proyecto de gestión de inventario de hardware

**Lorenzo, Ángel, Lanzarote y Alex**

**Tutor: Abraham**

Fecha de entrega: indefinida

**1º de Administración de Sistemas Informáticos en Red**

# Índice

**Introducción**

**Herramientas del proyecto**

**Objetivos del proyecto**

**Requisitos del proyecto**

**Planificación**

**Desarrollo del proyecto**

Tecnologías/Herramientas empleadas

Documentación técnica

**Conclusiones**

Resultados obtenidos y posibles mejoras futuras

6.1.1. Inventario del Almacén de Hardware

6.1.2 Desarrollo e Implementación de la Base de Datos

6.1.3. Diseño en el ámbito Web

6.1.4. Soporte Técnico y Configuración de Sistemas

**CE2A Clasificación y Versiones**

**CE2B 8. Árbol DOM**

**CE2D Comparativa HTML vs. XHTML**

**CE2E XHTML en Gestión de Datos**

**CE2G 7. Ventajas de usar .CSS**

1. Optimización del Rendimiento, Eficiencia y Carga de Archivos
2. Mantenimiento Centralizado y Arquitectura de Código
3. Estructura Semántica, Accesibilidad y SEO
4. Flexibilidad, Adaptabilidad y Diseño Responsivo
5. Cumplimiento de Estándares Oficiales

**Cuestionario de Preguntas Teórico-Prácticas Marta**

Sobre la evolución web (CE2A)

Sobre el analisis del codigo (CE2B, CE2C)

Sobre la rigurosidad sintáctica (CE2D, CE2E)

Sobre el diseño y presentación (CE2G, CE2H)

# Introducción

Tenemos que crear una forma de gestionar un almacén de hardware que nuestros profesores nos van a dar.

# Herramientas del proyecto

Para realizar este proyecto hemos utilizado las herramientas de Git/Github, Neocities, SQLite, Balsamiq, Magnific y Visual Code Studio.

# Objetivos del proyecto

Crear un inventario para gestionar un almacén, incluyendo una base de datos, una página web y documentación.

# Requisitos del proyecto

Un equipo funcional, un inventario digital, documentación técnica, justificación de las compatibilidades, un sistema operativo instalado y configurado, una base de datos, una propuesta de transformación digital y una memoria del proyecto.

# Planificación

A lo largo del proyecto, al grupo se nos ha asignado diferentes retos, cuestiones o problemas

**Dia 1:** Instalar Ubuntu (Lorenzo)

> (Problemas: El sistema operativo (Ubuntu) pide Usuario y Contraseña, Solucion: Reinstalar el sistema operativo de 0)

**Dia 2:** Desmontar el equipo y realizar inventario de los componentes, instalar ventoy en el USB (Realizado en colaboración del grupo)

> (Problemas: Las conexiones del panel frontal no se conectaron de manera correcta, Solución: Alex ha revisado las conexiones del panel frontal y las ha reordenado de manera correcta, permitiendo que el ordenador se encienda perfectamente)

**Dia 3:** Clonar repositorio del proyecto en Github, Prácticas individuales de HTML.

> (Problemas: No estábamos familiarizados con github, Solución: Prueba y error, finalmente Lorenzo tuvo que usar el CMD, para clonar el repositorio, submodular y subirlo a github a traves de git)

**Dia 4:** Limpieza de código HTML, Añadir nuevas funciones al HTML del inventario y implementacion de CSS para la IU (Alex), Reorganización de base de datos (Angel y Lanzarote)

> (Problemas: No hemos podido pasar de drawdb a sqlite, no hubo una solución clara, acabamos haciendo todo manualmente desde 0.)

**Dia 6:** Implementar pagina del inventario en Github/Neocities (Alex y Lorenzo), Revisar si el sistema funciona perfectamente

> (Problemas: Ninguno)

**Dia 7:** Implementación de Hipervínculos regresivos y otras funciones CSS en Neocities (Lorenzo) Realización del Mockup de la web (Angel y Lanzarote), Implementación de efecto “wow” en la web (animacion de intro, vínculos en la página web, etc.)

> (Problemas: Ninguno)

**Dia 8:** Continuación del trabajo previo sin terminar (En grupo)

> (Problemas: Ninguno)

**Dia 9:** Continuación del mockup en Balsamiq (Angel y Lanzarote), Continuación del Documento (Alex y Lorenzo)

> Problemas: terminamos la documentación expuesta en un formato diario y no como la plantilla de informe planteada por lo el dia 10 se procede al cambio.

**Dia 10:** Realización del documento correcto a petición de Marta (Lorenzo y Alex)

> (Problemas: Ninguno)

# Desarrollo del proyecto

## Tecnologías/Herramientas empleadas

Las tecnologías utilizadas han sido las siguientes:

- Magnific
- Balsamiq
- GitHub (enlace al github en el que está toda la información)
- Drawdb
- SQLite (La carpeta de instalación se encuentra en el google Drive)
- Neocities

## Documentación técnica

La documentación técnica utilizada en la búsqueda de información fue la información que aparecía en la página oficial de PcComponentes, Amazon España, The retro web, algunos manuales de instalación, PROComponentes, technical.city, dragonsteelmods.com, www.frostytech.com.

A continuación se pueden ver los enlaces a los distintos componentes en los que hemos sacado la información:

- Intenso Top Performance SSD M.2 2280 256GB SATA3
- Disco Duro WD Caviar Blue 500GB HDD 7200rpm SATA III 3.5"
- Samsung SpinPoint F3, modelo HD502HJ de 500GB
- Disco Duro SanDisk SSD Plus 240GB SATA III
- Athlon 3000G with Radeon Vega Graphics
- INTEL core 2 (no se puede distinguir el modelo específico)
- AMD Athlon II X2 250
- TACENS ANIMA APSI500 ATX 500W
- Thermaltake TR2 S 700W ATX
- Maxima Oke 450W ST-452
- fuente de alimentación PC CASE 500W
- ASUS EN9600GT
- sapphire ati radeon hd 3650
- asus the retro web eah3450/htp/512m/a
- Placa Base Gigabyte A520M K V2
- ASUS M2N68-AM PLUS
- ASUS P5LD2 SE
- Tarjeta de red inalámbrica interna (WiFi)
- Conceptronic C480i
- Tarjeta de red inalámbrica interna
- Tarjeta controladora interna de puertos de alta velocidad FireWire 400 (IEEE 1394a / i.LINK)
- Refrigeración por aire para CPU AMD Wraith Stealth
- Adata XPG GAMMIX D35 DDR4
- Ram Kingston DDR2 2 gb 800 Mhz
- Ram Samsung DDR4 4gb 2133 Mhz
- Ram Kingston DDR2 1gb 667 Mhz
- Ram Kingston DDR2 2gb 667 Mhz
- Refrigeración por aire para CPU Akasa EVO 120 (AK-920)
- Ventilador de chasis independiente (Case Fan) sin escobillas Ning Jie Electronic NJ12025SE
- Refrigeración por aire para CPU Gigabyte G-Power Cooler Pro

# Conclusiones

## Resultados obtenidos y posibles mejoras futuras

### 6.1.1. Inventario del Almacén de Hardware

- **Inventariado físico completo:** Se logró desmontar el equipo asignado, identificar cada uno de sus componentes de hardware y registrar toda la información para dar inicio al control del almacén y analizando compatibilidades de los distintos componentes con las placas bases de nuestro almacén.

### 6.1.2 Desarrollo e Implementación de la Base de Datos

- Hemos empezado a crear el inventario en una base de datos en SQLite al nivel y conocimiento de reto en la actualidad.

### 6.1.3. Diseño en el ámbito Web

- Se elaboró el Mockup de la web utilizando *Balsamiq*, asegurando una planificación visual antes del desarrollo.
- Se programó el sitio web utilizando código en HTML y CSS.
- **Efecto "Wow"** Se integraron elementos dinámicos atractivos, como una animación de introducción y vínculos interactivos dentro de la página.
- **Publicación:** La página web del inventario se subió en la plataforma de GitHub para su acceso público.

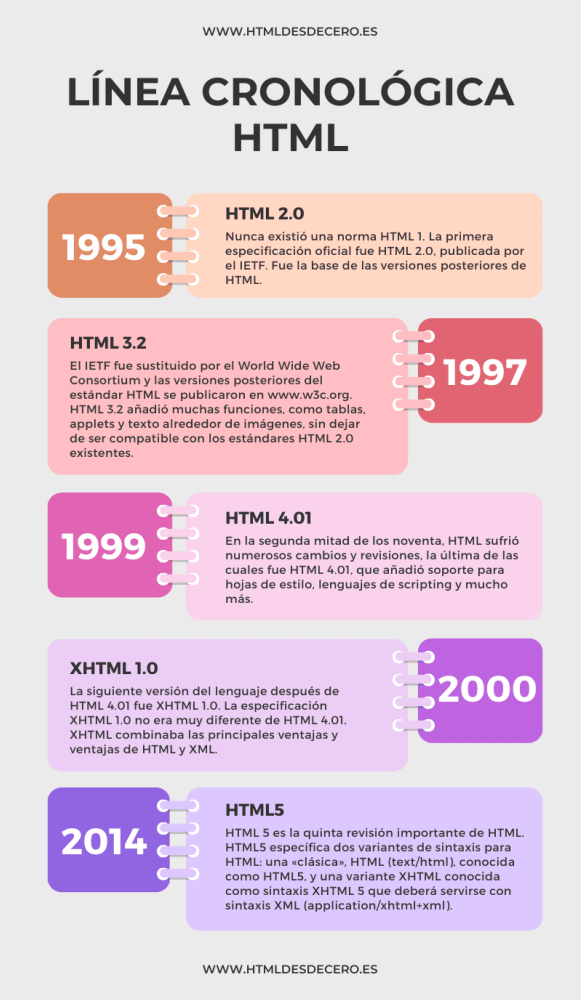
### 6.1.4. Soporte Técnico y Configuración de Sistemas

- **Puesta a punto del Sistema Operativo:** Se soluciona el bloqueo de credenciales inicial mediante la reinstalación limpia desde cero de Ubuntu, dejando el equipo completamente operativo.
- Se configuró un almacenamiento USB multiarranque utilizando *Ventoy*.

# CE2A Clasificación y Versiones

> *¿Has identificado y clasificado en la memoria técnica los lenguajes de marcas relacionados con la Web y la evolución de sus distintas versiones (ej. HTML4, XHTML, HTML5)?*

El HTML 1.0 (de 1991 a 1993) fue la primera versión de HTML, que solo tenía funcionalidades básicas. En 1995, HTML 2.0 se estandarizó oficialmente. En 1997, HTML 3.2 incorporó las tablas y mejoró la presentación de los documentos. HTML 4.0 (1997) y 4.01 (1999) fomentaron la separación entre HTML y CSS. XHTML 1.0 se creó en 2000 como una versión más estricta de HTML, basada en XML. Sin embargo, XHTML 2.0 se acabó descontinuando debido a su rigidez, las dificultades de implementación y la falta de compatibilidad con las páginas existentes. HTML5 comenzó a desarrollarse en 2004 y se publicó oficialmente en 2014, incorporando elementos semánticos, soporte para audio y vídeo y nuevas funcionalidades. HTML 5.1 (2016) y HTML 5.2 (2017) introdujeron nuevas mejoras. Desde entonces, HTML se mantiene como un estándar vivo (*HTML Living Standard*), actualizado continuamente por WHATWG.



# CE2B 8. Árbol DOM

```
document
└── html
    ├── head
    │   ├── meta, title
    │   └── link (stylesheet)
    └── body
        ├── aside (.intro-screen)
        │   └__section(intro-content)
        ├── header (.site-header)
        │   └__section  (header-content)
        │       └__section  (header-text)
        ├── nav (.site-nav)
        ├── main (.inventory-area)
        │   └__section (team-section)
        │       └__ ul (team-list)
        │    ├── section (componentes)
        │       └__ Table
        │           └__ Thead
        │           └__ Tbody
        │               └__ Tr (Procesador)
        │               └__ Tr (Memoria RAM)
        │               └__ Tr (Placa base)
        │               └__ Tr (Fuente de alimentación)
        │               └__ Tr (Disco)
        │               └__ Tr (Ventilador)
        │               └__ Tr (Cable de alimentación)
        │               └__ Tr (Bateria Bios)
        └── footer (.site-footer)
            └__ Section (footer-content)
                └__ section (footer-text)
```

# CE2D Comparativa HTML vs. XHTML

> *¿Diferencias claramente en la memoria o exposición de las semejanzas y reglas sintácticas que diferencian los lenguajes HTML y XHTML?*

HTML es más permisivo frente a XHTML

- Te permite dejar muchas etiquetas sin cerrar (Aunque esta no sea la práctica correcta)
- Las etiquetas vacías se pueden dejar sin cerrar (En XHTML, siempre hay que cerrarlas)
- HTML te permite escribir las etiquetas en mayúscula, mientras que XHTML solo permite escribir en minúscula, dando un error si esto no ocurre
- Si no hay espacios, en HTML puedes poner atributos sin las comillas, por ejemplo: <input type=text> en vez de <input type=”text”>
- XHTML al ser código XML, no permite errores y al tener fallos, la página web simplemente muestra un error, mientras que HTML al tener errores activa un modo de compatibilidad que “adivina” el contenido de la página web

# CE2E XHTML en Gestión de Datos

> *¿Justificas la utilidad de la sintaxis estricta de XHTML cuando se integran páginas web con sistemas de gestión de información, inventarios y bases de datos?*

Aunque XHTML ofrece una sintaxis más estricta que facilita la validación del código y puede favorecer la automatización de ciertos procesos, consideramos que HTML5 es una opción más adecuada para el desarrollo web moderno, sobre todo para nuestro proyecto.

HTML5 es más flexible y tolerante a errores sintácticos, lo que facilita el desarrollo, el mantenimiento y la actualización de las páginas web. Además, permite integrar formularios, tablas y elementos interactivos que pueden utilizarse para mostrar y gestionar información procedente de inventarios y bases de datos.

Es importante destacar que la conexión con bases de datos y la automatización de procesos dependen principalmente de las tecnologías del servidor, las API y los lenguajes de programación utilizados, y no exclusivamente de la sintaxis del documento HTML. Por tanto, HTML5 también permite desarrollar sistemas automatizados sin necesidad de aplicar las restricciones sintácticas de XHTML.

# CE2G 7. Ventajas de usar .CSS

## 1. Optimización del Rendimiento, Eficiencia y Carga de Archivos

- **Reducción del tamaño de los documentos HTML:** Al eliminar los atributos de diseño obsoletos el código HTML se vuelve mucho más limpio y ligero reduciendo drásticamente el peso de los archivos.
- **Aprovechamiento del almacenamiento en caché :** El archivo CSS externo (vinculado mediante la etiqueta <link>) se descarga una sola vez durante la primera visita. En las siguientes páginas, el navegador utiliza el archivo guardado en caché acelerando notablemente el tiempo de carga del sitio.

## 2. Mantenimiento Centralizado y Arquitectura de Código

- **Mantenimiento centralizado y eficiencia en el desarrollo:** Nos permite guardar el estilo y colores corporativos en único archivo .css para que la modificación se replique instantáneamente tantas páginas HTML como queramos, eliminando la necesidad de repetir código de diseño.
- **Desarrollo en paralelo:** Permite trabajar simultáneamente de forma independiente; el programador puede estructurar los datos en el HTML mientras el diseñador define el aspecto visual en el CSS.

## 3. Estructura Semántica, Accesibilidad y SEO

- **Mejora del posicionamiento en buscadores (SEO):** Al estar el HTML libre de código de presentación, los motores de búsqueda (como Google) rastrean e indexan el contenido relevante de forma mucho más rápida y eficiente, mejorando significativamente la densidad de código frente a texto.
- **Mayor accesibilidad web multidispositivo:** Los lectores de pantalla para personas con discapacidad visual leen el código estructurado secuencialmente. Al separar presentación, se garantiza que estos dispositivos interpreten correctamente el contenido (encabezados, párrafos, listas) facilitando el uso.

## 4. Flexibilidad, Adaptabilidad y Diseño Responsivo

- **Independencia del dispositivo y factores de forma:** Permite adaptar la visualización de un mismo contenido a diferentes dispositivos (pantallas de escritorio, smartphones, tabletas o incluso versiones de impresión).
- **Herramientas de reestructuración dinámica:** Mediante el uso de herramientas como Unidades del Viewport . Container Querie , *Flexbox* …. Es posible reconfigurar por completo la disposición espacial de la página web sin necesidad de tocar ni una sola línea del código HTML base.

## 5. Cumplimiento de Estándares Oficiales

- **Cumplimiento de los estándares oficiales del W3C:** El Consorcio W3C ha declarado formalmente obsoletos los elementos y atributos de marcado destinados únicamente a la presentación. El uso de CSS es el estándar oficial exigido para definir el aspecto visual en la web moderna.

# Cuestionario de Preguntas Teórico-Prácticas Marta

## Sobre la evolución web (CE2A)

> *¿Qué necesidades técnicas impulsaron la transición desde las primeras versiones de HTML hacia XHTML y el posterior desarrollo de HTML5?*

La transición desde las primeras versiones de HTML hacia XHTML y, posteriormente, HTML5 estuvo impulsada por la necesidad de mejorar la estandarización, la compatibilidad entre navegadores y la integración de nuevas tecnologías web.

- **HTML:** Permitía crear páginas web, pero los navegadores interpretan el código de forma diferente y toleraban errores, dificultando su mantenimiento.
- **XHTML:** Introdujo las reglas estrictas de XML, exigiendo un código bien estructurado para facilitar su validación y procesamiento.
- **HTML5:** Surgió para adaptarse a la web moderna, incorporando etiquetas semánticas, audio, vídeo y gráficos, además de reducir la dependencia de complementos externos como Flash.

## Sobre el analisis del codigo (CE2B, CE2C)

> *Muestra un fragmento de tu código HTML del inventario e identifica la jerarquía de al menos tres niveles de nodos en el árbol DOM, explicando el propósito de cada etiqueta y atributo utilizado.*

(A partir de la línea: 67)

`<table>` - **Nivel 1** Tabla completa del inventario `L: 67`

- `<tbody>` - **Nivel 2** Cuerpo de la tabla `L: 77`
  - `<tr>` - **Nivel 3** Fila correspondiente al procesador `L: 79`
    - `<td>` - **Nivel 4** Celda con características `L: 82`
      - `<ul>` - **Nivel 5** Lista de características `L: 83`
        - `<li>` - **Nivel 6** Cada característica individual `L: 84`

## Sobre la rigurosidad sintáctica (CE2D, CE2E)

> *¿Por qué un documento XHTML mal formado puede fallar al conectarse con una base de datos o sistema de inventario, mientras que HTML5 suele ser más permisivo con los errores del desarrollador?*

Un documento XHTML mal formado puede provocar errores porque XHTML exige una sintaxis estricta: las etiquetas deben cerrarse correctamente, los elementos deben anidarse de forma adecuada y los atributos deben escribirse siguiendo las reglas de XML. Si no se cumplen estas normas, el documento puede no procesarse correctamente y dificultar la comunicación con otros sistemas, como una base de datos o un sistema de inventario.

En cambio, HTML5 es más permisivo y permite que los navegadores corrijan automáticamente muchos errores de sintaxis, por lo que la página puede seguir mostrándose aunque el código no sea perfecto.

## Sobre el diseño y presentación (CE2G, CE2H)

*¿Qué problemas surgirían al mantener la web del almacén si aplicaras los estilos visuales directamente con atributos HTML incrustados en lugar de utilizar un archivo CSS externo?*

Los principales problemas serían la dificultad para mantener y actualizar la web, ya que habría que modificar los estilos directamente en el código HTML, buscando cada atributo que se quisiera cambiar. Por ejemplo, si se quisieran actualizar los colores corporativos, habría que localizar y modificar cada aparición de esos colores.

En cambio, al utilizar un archivo CSS externo, bastaría con modificar los estilos en un único lugar para que los cambios se aplicaran automáticamente a todas las páginas HTML que utilizaran esa hoja de estilos.